ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

# On Combiners for Verifiable Random Functions

Bachelor Thesis

Basil Rohner

March 17, 2026

Advisors: Prof. Dr. A. D. Visor, Dr. P. Ostdoc
Department of Computer Science, ETH Zürich

# Acknowledgments

I would like to thank my supervisors Nicholas Brandt and Xiaoqi Duan for their continuous support and patience. I also want to thank Professor Dennis Hofheinz for the opportunity to write my thesis at his group. Finally, I would like to thank my family and friends for their endless support and encouragement.

*"The wise man does at once what the fool does finally."*

— Niccolò Machiavelli

**Abstract**

Verifiable random functions (VRFs) are an extension of pseudorandom functions (PRFs), one can evaluate the function using a secret key to obtain random-looking images and corresponding proofs. Given the corresponding public key one can verify whether a given image was generated from an input if given a suitable proof.

In this thesis we study ways to combine multiple instances of VRFs in a black-box fashion. Specifically we study the existence of robust black-box VRF-combiners, constructions which are given oracle access to a set of candidate schemes, of which a sufficiently large (unknown) subest implements the VRF securely, and implements a secure VRF.

While there exist simple constructions of such robust black-box combiners for the related PRF primitive even if only one candidate implements a secure PRF, we show that there exists no robust black-box VRF-combiners if not at least half of the candidate schemes are secure, even if we relax the required security properties from the combiner.

# Contents

**Contents** **iv**

**1 Introduction** **1**
1.1 Motivation and Previous Work . . . . . 1
1.2 PRF Combiners . . . . . 3
1.3 Our Contribution . . . . . 4

**2 Notation and Preliminaries** **7**
2.1 Notation . . . . . 7
2.2 Verifiable Functions . . . . . 8
2.3 Robust Combiners . . . . . 11

**3 Results** **13**
3.1 Robust Black-Box DAG Combiner . . . . . 13
3.2 Impossibility of Robust Black-Box VRF-Combiner . . . . . 18
3.3 Strong Robust Black-Box Combiners . . . . . 35
3.4 Discussion . . . . . 38

**4 Conclusion** **39**

**Bibliography** **40**

**Appendix** **41**
The Difference Lemma . . . . . 41
Conjecture . . . . . 41

Chapter 1

# Introduction

## 1.1 Motivation and Previous Work

**Pseudorandom Functions.** A pseudorandom function (PRF) [GGM86] is a deterministic algorithm that takes as input a secret key $k$ and a point $x$ to produce an output $y$ that looks random in the sense that no efficient adversary, modeled by a probabilistic polynomial time (ppt.) Turing machine can distringuish the output $y$ from a random bitstring $r$ of the same length. This must hold even if the adversary has access to a polynoial number of point evaluations $(x, y)$ before distinguishing the output. PRFs are fundamental to modern cryptography and used to construct ....

**Verifiable Functions.** A verifiable function (VF) is a public-key primitive that allows the holder of a secret key $\mathsf{sk}$ to evaluate points $x$ to obtain image-proof pairs $(y, \pi)$ such that any party holding the public key $\mathsf{pk}$ can verify that $y$ was indeed computed correctly from $x$ if given the proof $\pi$. We require correctness and unique provability from the verification which state that correctly generated image-proof pairs $(y, \pi)$ always (or except with negligible probability) is accepted by the verification algorithm and that at most one image can be proven respectively.

In this work we primarily study extensions of VRs, primarily the verifiable random function (VRF) introduced by Micali, Rabin and Vadhan in [MRV99] that additionally requires the outputs $y$ to be pseudorandom to efficient adversaries without access to the corresponding proof. Similar to the pseudorandomness definition for PRFs we require that no efficient adversary can distinguish the images $y$ from random outputs $r$, even if the adversary gets to see polynomially many evaluations of $x$ with their corresponding proof. VRFs can be seen as a public-key analogoue of PRFs with verifiability requirements.

In practice VRFs are used to implement signatures with strong verifiability guarantees [**?**], resettable zero-knowledge proofs [**?**], lottery and e-cash systems [**?**] and they play a big role in distributed leader election protocols that runs many Proof-of-Stake blockchain systems [**?**].

Additionally we study VFs satisfying weaker notions of pseudorandomness such as weak pseudorandomness and unpredictability which give rise to verifiable unpredictable functions (VUFs) [**?**] and weak verifiable random functions (wVRFs) [**?**] respectively. VUFs additionally require that the evaluation of the VF on a fresh evaluation point cannot be predicted by efficient adversaries except with negligible probability and wVRFs require that pseudorandomness holds, but only on uniformly random evaluation points $x$, instead of a point chosen by the adversary.

add stronger notions cVRF, khVRF, aggVRF, unbiasVRF if time

**Robust Combiners.** In cryptography we often take a reductionist approach when constructing new cryptographic primitives, in general constructions satisfying computational security properties rely on cryptographic hardness assumptions. To hedge against the breakdown of cryptographic hardness assumptions, and thus the subsequence failure of the construction we may want to base the security of systems on multiple hardness assumptions. Robust combiners [**?**] are constructions that combine $n$ candidate schemes, concrete instances of the same primitive, and implement a secure instance of that primitive if a large enough subset of those candidate is secure. The notion of combiners has gained traction with the rise of post-quantum and hybrid-cryptography areas

Stronger version of robust combiners are robust black-box combiners [**?**] which are only given query access to the candidate schemes as oracles, i.e. cannot inspect the Turing machine of the candidate scheme. The motivation of a robust black-box combiner is to have a combiner that can implement a secure scheme only with access to sub-schemes with the underlying code of the candidate schemes are not accessible to the combiner. We wish for the combiner to derive its security from the evaluation of the secure subset of oracles, not by making any modifications to the candidates or simply implementing a secure instance of the primitive himself. For that reason we additionally require robust black-box combiners to implement the secure instance relative to a PSPACE complete oracle, that is guarantee the security against adversaries with access to a PSPACE oracle.

Robust black-box combiners have been considered for a variety of cryptographic primitives such as oblivious transfer [**?**], non-interactive zero-knowledge proofs (NIZK) [**?**], non-interactive witness-indistinguishable arguments (NIWI) [**?**], indistinguishability obvuscation (iO) [**?**] and signature schemes [**?**]. Additionally many folklore constructions exist for PRF, key exchange protocols . While there exist VRF constructions rely on PRFs and

add some

NIWIs, for both of which combiners have been studied, black-box combiners for VRFs have, to the best of our knowledge, not been studied.

## 1.2 PRF Combiners

VRF can be considered as an extension of PRFs to the public-key setting with the additional requirements of verifiability. Simple robust black-box PRF combiners exist. One combiner works by evaluating the evaluation point $x$ with every candidate scheme to obtain images $y_1, y_2$ and returning the XOR of the two images $y = y_1 \oplus y_2$. Pseudorandomness of the combined scheme can be shown with a reductionist approach. Specifically we show that given an adversary against the combined scheme, we can construct an adversary against every candidate scheme running in the same time with the same advantage. By contraposition, assuming that at least one candidate scheme is pseudorandom and there exists no efficient combiner with non-negligible advantage against the candidate we show that there exists no efficient adversary against the combined scheme with non-negligible advantage. We illustrate the construction of a robuts $(2;1)$ black-box PRF-combiner in Section 1.2.

$\underline{\mathsf{Eval}_{\mathsf{PRF}}^{\mathcal{O}_1,\mathcal{O}_2}(k,x)}$

1 : $y_1 \leftarrow \mathcal{O}_1(k,x)$
2 : $y_2 \leftarrow \mathcal{O}_2(k,x)$
3 : **return** $y_1 \oplus y_2$

**Figure 1.1:** Robust $(2;1)$ Black-Box PRF-combiner.

We outline the reduction and refer to the appendix for the formal proof. The reduction against candidate scheme $\mathcal{O}_i$ relies on the pseudorandomness oracle $\mathcal{O}_{eval}$ and simulates the other candidate $\mathcal{O}_{2-i}$ himself to answer queries of the combiners adversary with $\mathcal{O}_{eval}(x) \oplus \mathcal{O}_{2-i}(k,x)$. The adversary against the combined scheme then guesses the hidden bit $b$, trying to distinguish the random world from the real world which the reduction then uses to guess its own hidden bit $b'$. If the hidden bit $b' = 0$ then the adversary $\mathcal{A}$ is simulated perfectly in the sense that the reduction wins the security experiment with the same advantage of $\mathcal{A}$ while for bit $b' = 1$ the answers to the queries of the adversary are still uniformly random bitstrings and therefore the reduction also wins with the same probability.

Note that the existance of a robust $(2;1)$ black-box PRF-combiner naturally leads to a robust $(n;1)$ black-box VRF-combiner for any natural number $n$ by iteratively applying the construction on the construction and a new candidate. Since there exists a robust $(n;1)$ black-box PRF-combiner one could hope

for a similar construction to work for VRFs. This is however not the case, as we will show in Theorem 3.17 there exist no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner for VRFs.

## 1.3 Our Contribution

**Robust Black-Box DAG Combiner.** We introduce the notion of robust black-box DAG combiners, formally defined in Definition 3.1, DAG combiners for short. DAG combiners are black-box combiners which are defined by directed acyclic graphs (DAGs) for every implemented algorithm. The implemented algorithms are specified through computation that runs along the DAG starting from a designated source vertex and ending in a target vertex. In the execution vertices correspond to queries to implementation functions of candidate oracles on an input computed from all preceding vertices by a ppt. Turing machine (without oracle access to the candidates). DAG combiners make the structure of the black-box combiner more explicit by structuring it into interleaved oracle queries to the candidats and intermediate computation, that we model with Turing machines without oracle access to the candidates. We illustrate this concept in Figure 1.2 on the example of the simple robust black-box PRF-combiner in Section 1.2.

To connect DAG combiners and general black-box combiners we show that the notions are indeed equivalent, that is, that for every general black-box combiner we can construct a DAG combiner that behaves identically in the sense that the algorithms defined by the combiners are equivalent. We state this result formally in Theorem 3.2. The concept of a robust black-box DAG combiner allows us to reason over a more structured representation of robust black-box combiners, more similar to a circuit in structure simplifying the proofs later on.

**Robust Black-Box VRF-Combiner.** Following the introduction of a robust black-box DAG combiner, we study the simplest robust black-box VRF-combiner, robust $(2;1)$ black-box VRF-combiners. We answer the question on whether there is a construction that combines two candidates, of which only at least one is secure, negatively. Specifically we show impossibility of robust $(2;1)$ black-box VRF-combiners in **??**.

Since we can naturally use a robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner to construct a $(2;1)$ combiner by simply initializing half the candidates with $\mathcal{O}_1$ and the other half with $\mathcal{O}_2$ we get an impossibility result for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner by contraposition. Next we inspect robust black-box combiners with additional assumptions on the candidate schemes and relaxed requirements on the construction in the hopes of finding combiner constructions. We show that even if we only require unpredictability, sta-

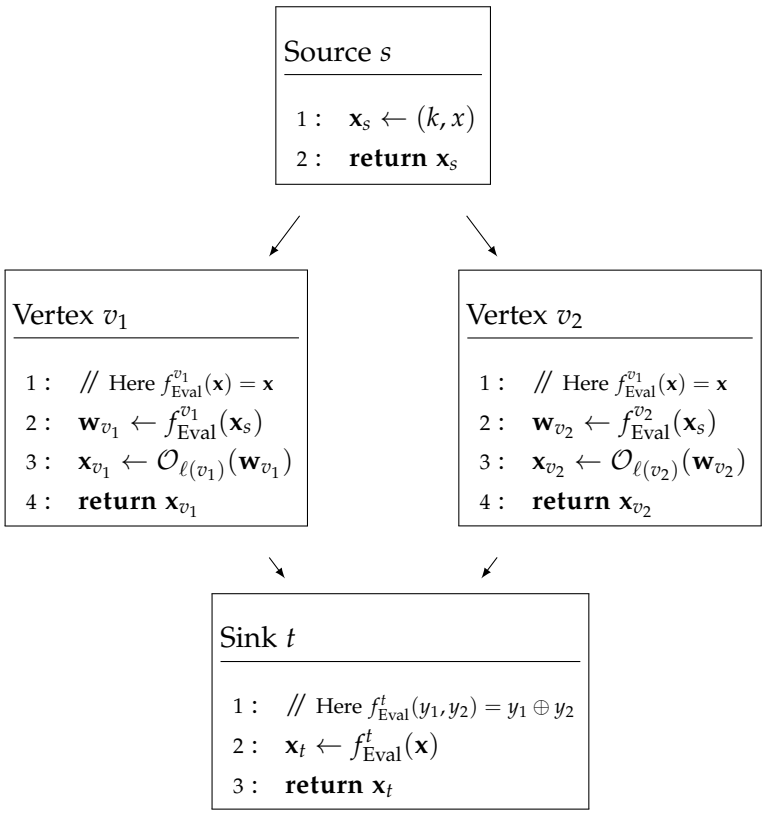



**Figure 1.2:** Robust $(2;1)$ Black-Box DAG PRF-combiner. Note that for every node we first combine the output of all predecessors using a ppt. Turing machine $f^{v}_{\text{Eval}}$. The output $\mathbf{w}_v$ of $f^{v}_{\text{Eval}}$ is then queried to the corresponding oracle $\mathcal{O}_{\ell(v)}$. The source vertex doesn't compute anything but simply outputs the inputs while the sink $t$ doesn't query any oracle and simply combines its inputs.

tistical correctness and statistical unique provability from the construction, the impossibility holds Theorem 3.17, this gives two corollaries excluding the possibility of robust $(n; \lfloor n/2 \rfloor)$ black-box combiners for wVRF and VUF in Corollary 3.18 and Corollary 3.19. By strengthening the assumptions on the candidates for all candidates to implement pseudorandomness is also insufficient to find constructions Theorem 3.17.

unclear

We show that if we assume all candidates to be uniquely provable and perfectly correct, the combination of the candidates works in a very similar fashion to the combiner on the PRF primitive. Note that since unique provability and perfect correctness are generally information theoretic properties, this means that for most VRF constructions, e.g. LatticeVRF [**?**] and ECVRF [**?**], those properties are typically proven mathematically. Arguable combiners are only interesting for combining computational security properties that

rely on hardness assumptions.

Having strengthened the assumptions on the candidates we can now consider more advanced notions of VRFs. We'll show that with simple extensions we can construct combiners for constrainable VRFs, key-homomorphic VRFs and aggregable VRFs if all candidates are guaranteed to be uniquely provable and perfectly correct. Finally we consider unbiasable VRFs for which we provide a combiner that relies on collision resistant hash functions and deterministic 2-source randomness extractors.

Chapter 2

# Notation and Preliminaries

## 2.1 Notation

For a positive integeger $n$ define $[n] = \{1, \ldots, n\}$. We use $\lambda \in \mathbb{N}$ exclusively to denote the security parameter. We use $\mathsf{poly}(\lambda)$ to denote the set of functions that are polynomial in $\lambda$ in the sense of Definition 2.1 and $\mathsf{negl}(\lambda)$ to denote the set of functions that are negligible in $\lambda$ in the sense of Definition 2.2. We call a function $f$ *overwhelming* if $1 - f(\lambda) \in \mathsf{negl}(\lambda)$. We use $\leftarrow$ for simple assignment. For a set $S$ we write $s \leftarrow_\$ S$ to indicate that the value $s$ is sampled uniformly at random from set $S$ and for a probabilistic algorithm $\mathcal{A}$ we write $y \leftarrow_\$ \mathcal{A}(x)$ to denote assigning to $y$ the output of $\mathcal{A}$ on input $x$. We use ppt. to abbreviate probabilistic polynomial time in the context of Turing machines. Finally, we use bold letters, e.g. $\mathbf{x}$, to indicate that $\mathbf{x}$ consists of multiple values such as tuples or vectors.

maybe change

**Definition 2.1 (Polynomial Functions)** *We say that a function $p : \mathbb{N} \to \mathbb{R}$ is polynomial if the folowing holds*

$$\exists d \in \mathbb{N}, \exists n_0 \in \mathbb{N}, \forall n > n_0, p(n) \leq n^d$$

**Definition 2.2 (Negligible Functions)** *We say that a function $\varepsilon : \mathbb{N} \to \mathbb{R}$ is negligible if the following holds*

$$\forall p \in \mathsf{poly}(\lambda), \exists n_0 \in \mathbb{N}, \forall n > n_0, |\varepsilon(n)| < \frac{1}{p(n)}$$

## 2.2 Verifiable Functions

A verifiable function (VF) VF is a cryptographic primitive that implements algorithms $\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}$ and $\mathsf{Vfy}_{\mathsf{VF}}$. A user can generate key-pairs $(\mathsf{sk}, \mathsf{pk})$ using $\mathsf{Gen}_{\mathsf{VF}}$ consisting of a secret key sk and a public key pk, use the secret key sk to make evaluate arbitrary points $x \in \{0,1\}^\lambda$ using $\mathsf{Eval}_{\mathsf{VRF}}$ and use the public key pk to verify a triple $(x, y, \pi)$ using $\mathsf{Vfy}_{\mathsf{VRF}}$. We give a formal definition in Definition 2.3, the definition takes inspiration of the definition of a VRF in [**?**]. To reduce redundancy we define additional security properties such as pseudorandomness and unpredictability seperately.

**Definition 2.3 (Verifiable Function)** *A verifiable function (VF)* VF *is a triple of efficient (ppt.) algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *with the syntax*

this is less elegant now

- $\mathsf{Gen}_{\mathsf{VF}}$*: The key-generation algorithm takes as input the unary encoding of the security parameter and produces a pair* $(\mathsf{sk}, \mathsf{pk}) \in \mathcal{SK} \times \mathcal{PK}$ *containing a secret and a public key respectivley.*
- $\mathsf{Eval}_{\mathsf{VF}}$*: The evaluation algorithm takes as input a tuple* $(\mathsf{sk}, x)$ *containing a secret key* $\mathsf{sk} \in \mathcal{SK}$ *and an evaluation point* $x \in \{0,1\}^\lambda$ *and produces an tuple* $(y, \pi)$ *containing an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$*.*
- $\mathsf{Vfy}_{\mathsf{VF}}$*: The verification algorithm takes as input a tuple* $(\mathsf{pk}, x, y, \pi)$ *containing a public key* $\mathsf{pk} \in \mathcal{PK}$*, an evaluation point* $x \in \{0,1\}^\lambda$*, an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$ *and returns a bit b denoting acceptance or rejection.*

can evaluation or verification be randomized, probably vfy can be probabilistic

Note that a VF is a purely syntactical concept. For a VF we define various properties to arrive to arrive at specific cryptographic primitives, e.g. VRFs. First of all we define $\varepsilon$-correctness, a property that guarantes that over random key-pairs and evaluation points, the verification algorithm accepts a correctly computed image-proof pair with probability at least $\varepsilon$.

**Definition 2.4 ($\varepsilon$-Correctness, [?])** *A verifiable function* $\mathsf{VF} = (\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is $\varepsilon$-correct, if for all security parameters $\lambda \in \mathbb{N}$ we have that*

$$\Pr\left[\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = 1 : \begin{array}{c}(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda) \\ x \leftarrow_{\$} \{0,1\}^\lambda \\ (y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)\end{array}\right] \geq \varepsilon(\lambda)$$

*We define a VF* VF *to be perfectly correct, or just correct, if it is* 1*-correct and statistically correct if $\varepsilon$ is an overwhelming function.*

Intuitively a correct VF states that honestly generated image-proof pairs are accepted by the verification algorithm. Next we define unique provability. A VF is uniquely provable if not more than one image can be proven, even for maliciously crafted public keys.

**Definition 2.5 (Unique Provability, [?])** *A verifiable function* $\mathsf{VF}$*, with algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$*, is defined to be uniquely provable if for all evaluation points* $x \in \{0,1\}^\lambda$*, all public-keys* $\mathsf{pk} \in \mathcal{PK}$ *and any two images-proof pairs* $(y_0, \pi_0), (y_1, \pi_1) \in \mathcal{Y} \times \mathcal{PK}$ *we have:*

$$\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \wedge \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1$$

maybe comment why no statistical notion

Correctness and unique provability are information theoretic. We define pseudorandomness of a VF which is a computational property, stating that no efficient adversary $\mathcal{A}$ can distinguish images $y$ corresponding to evaluation of $x$, chosen by $\mathcal{A}$, from a uniformly random output $y^* \leftarrow_\$ \mathcal{Y}$. This needs to hold even if the adversary gets to see the evaluation of a polynoial number of points $x \neq x^*$.

**Definition 2.6 ($(t,\varepsilon)$-Pseudorandomness, [?])** *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* $(t,\varepsilon)$*-pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the prf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

*We say that a VF is pseudorandom, if it is* $(t,\varepsilon)$*-pseudorandom with* $\varepsilon$ *being a negligible function and* $t$ *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Next we define two weaker notions of pseudorandomness. While (strong) pseudorandomness required the evaluation of adverserially chosen points to be indistinguishable from uniformly random elements $y^* \in \mathcal{Y}$ we can relax this requirement by requiring computational indistinguishability for uniformly random challenges $x^* \leftarrow_\$ \{0,1\}^\lambda$. We call this notion weak pseudorandomness.

**Definition 2.7 ($(t,\varepsilon)$-Weak Pseudorandomness, [?])** *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* $(t,\varepsilon)$*-weakly pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the wprf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

*We say that a VF is weakly pseudorandom, if it is* $(t,\varepsilon)$*-weakly pseudorandom with* $\varepsilon$ *being a negligible function and* $t$ *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Alternatively a VF can be unpredictable. A VF is unpredictable if no adversary can predict the image $y$ of the evaluation of a challenge $x$ chosen by the adversary. This property needs to hold even if the adversary sees the evaluation of a polynomial number of points $x \neq x^*$. Note that if the adversary can predict the output of evaluation on $x^*$ he can also distinguish the output from a random output $y^* \leftarrow_{\$} \{0,1\}$ with non-negligible probability making unpredictability is a weaker property than pseudorandomness.

**Definition 2.8 ($(\varepsilon, t)$-Unpredictability, [?])** *A VF $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ is defined to be $(t, \varepsilon)$-unpredictable if for all security parameters $\lambda \in \mathbb{N}$, for all stateful adversaries $\mathcal{A}$, running in time at most $t(\lambda)$, the advantage of $\mathcal{A}$ in the up security experiment outlined in Figure 2.1 is at most $\varepsilon(\lambda)$, i.e. if*

$$\mathbf{Adv}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

*We say that a VF is unpredictable, if it is $(t, \varepsilon)$-unpredictable with $\varepsilon$ being a negligible function and $t$ being a polynomial, i.e. $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$.*

We now define the notions of a verifiable random function (VRF), verifiable unpredictable function (VUF) and weak verifiable random function (WVRF), which are correct, uniquely provable and pseudorandom, unpredictable and weakly pseudorandom VFs respectivly. We formally define the primitives.

**Definition 2.9 (Verifiable Random Functions)** *A verifiable random function $\mathsf{VRF} = (\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and pseudorandomness in the sense of Definition 2.6.*

**Definition 2.10 (Weak Verifiable Random Functions)** *A weak verifiable unpredictable function $\mathsf{WVRF} = (\mathsf{Gen}_{\mathsf{WVRF}}, \mathsf{Eval}_{\mathsf{WVRF}}, \mathsf{Vfy}_{\mathsf{WVRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and weak pseudorandomness in the sense of Definition 2.7.*

**Definition 2.11 (Verifiable Unpredictable Functions)** *A verifiable unpredictable function $\mathsf{VUF} = (\mathsf{Gen}_{\mathsf{VUF}}, \mathsf{Eval}_{\mathsf{VUF}}, \mathsf{Vfy}_{\mathsf{VUF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and unpredictability in the sense of Definition 2.8.*

Experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $\sigma \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $x^* \leftarrow_{\$} \{0,1\}^{\lambda}$
6 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
7 : $y_1 \leftarrow_{\$} \mathcal{Y}$
8 : $b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, x^*, \sigma, y_b)$
9 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
2 : $S \leftarrow \varnothing$
3 : $(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
4 : $(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
5 : **return** $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\text{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** $y$

**Figure 2.1:** The pseudorandomness security experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.6, the weak pseudorandomness security experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.7 and the unpredictability security experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.8. The main changes between the pseudorandomness security experiment and the weak pseudorandomness security experiment are highlighted in grey.

## 2.3 Robust Combiners

For two natural numbers $n, k$ we define the robust $(n;k)$ black-box combiner, or robust black-box combiner for short. A robust $(n;k)$ black-box combiner for a primitive $\mathsf{P}$ is a construction $\Pi_{\mathsf{P}}$, to be precise a ppt. oracle Turing machine, that is given query access to $n$ oracles, the candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$. If at least $k$ candidate schemes implement the primitive $\mathsf{P}$, then the combiner $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}$ securely implements $\mathsf{P}$. The idea behind a robust black-box combiner is that the construction extracts the security properties from the subset of correct candidates, even if some candidate schemes are not secure instances. However now note the following problem in the current definition:

If there exists a secure instance of $\mathsf{P}$ there always exists robust $(n;0)$ black-box combiner that simply ignores its candidates and implements $\mathsf{P}$. This robust black-box combiner meets all the requirements of the definition. Showing

that there exists no specific robust black-box combiner therefore requires to show that no secure instance of the primitive P exists, which is often a difficult complexity theoretic statement.

To circumvent this problem we want to explicitly force the combiner to derive its security from the subset of secure candidate schemes. To do this, we require the security properties of the combiner to hold relative to a PSPACE complete oracle. Firstly we want to note that this means that the adversary against the combiner can efficiently check whether an element is a member of a PSPACE complete language, enabling the adversary to generally break computational hardness assumptions but limiting the adversary still to a polynomial number of oracle queries. Secondly we want to note that this notion of a combiner still doesnt solve the issue if the security of the cryptographic primitive are *all* information theoretic.

Very unhappy with this definition

TODO state contrapositive of the definition below here.

**Definition 2.12 (Robust $(n;k)$ Black-Box P-Combiner)** *A robust $(n;k)$ black-box combiner for primitive* P *with k algorithms is a tuple of ppt. oracle Turing machines $\Pi_{\mathsf{P}} = (\mathsf{Alg}_{\mathsf{P}}^1, \ldots, \mathsf{Alg}_{\mathsf{P}}^k)$ such that if there exists an ppt. adversary $\mathcal{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_n,\mathsf{PSPACE}}$ that against $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ then there exist $n-k+1$ ppt. adversaries $\mathcal{B}_1^{\mathcal{O}_{\ell(1)}}, \ldots, \mathcal{B}_{n-k+1}^{\mathcal{O}_{\ell(n-k+1)}}$ and an injective function $\ell : [n-k+1] \to [n]$ such that for any $i \in [n-k+1]$ the adversary $\mathcal{B}_i^{\mathcal{O}_{\ell(i)}}$ breaks the security of the candidate $\mathcal{O}_{\ell(i)}$.*

**Remark.** If P defines composite security properties, as is the case for VRFs for instance, we say that an adversary breaks the security of P if it breaks one of the required security properties. For computational security properties such as pseudorandomness this typically means that the adversary wins the corresponding security experiment with non-negligible advantage. For information theoretic or statistical properties such as unique provability or correctness this means that the property doesn't hold (we can think of this as the adversary $\mathcal{A}$ being arbitrary, breaking the security).

Chapter 3

# Results

## 3.1 Robust Black-Box DAG Combiner

this still needs refactoring

Intuitively, a robust black-box combiner operates on $n$ candidate schemes by interleaving oracle calls to the candidates implemented functions with processing to combine their outputs in a way such that security properties of the secure subset of oracles are preserved if there are sufficiently many of them. To argue about the impossibility of such constructions it is helpful to impose additional structure on the combiner, to structure it as a directed acyclic graph modeling the computation of the combiner. In the DAG every oracle query corresponds to a vertex colored with the appropriate label Viewing a robust black-box combiner as a robust black-box DAG combiner is therefore simply a way make the structured execution of a combiner explicit, making the reasoning about oracle query dependencies and oracle-independent computation explicit. Additionally since robust black-box combiners need to satisfy the security properties relative to a PSPACE oracle, in the later proof we rely on the ability of the PSPACE oracle to brute-force many secret keys in the algorithm if there are not queries to oracles. For this line of reasoning this description is also more convenient. As we will show later in Theorem 3.2, the notions of robust black-box combiners and robust black-box DAG combiners are equivalent.

We define robust black-box DAG combiners by representing each implemented algorithm $\mathsf{alg}$ with a directed acyclic graph $\mathcal{G}_{\mathsf{alg}} = (V_{alg}, E_{alg})$ with a source $s_{\mathsf{alg}} \in V_{\mathsf{alg}}$ that has no in-going edges, corresponding to the input of the algorithm, and a sink $t_{alg} \in V_{alg}$ with no out-going edges, corresponding to a final processing step. Additionally the description of the DAG combiner contains a vertex labling $l_{alg} : V_{alg} \setminus \{s_{alg}, t_{alg}\} \to [n] \times \{alg_1, \ldots, alg_k\}$ which maps every vertex that is not a source or a sink to the corresponding candidate oracle and the specific function called on an input $\mathbf{x}_v$ to produce an output $\mathbf{y}_v$. We define the output $\mathbf{y}_{s_{alg}}$ of the source to be the combiner

Algorithm $[\![\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}; \rho]\!](\mathbf{x})$

$\mathbf{y}_s \leftarrow \mathbf{x}$
**for** $v \in \tau(\mathcal{G}, s) \setminus \{s, t\}$ **do**
  $\mathbf{x}_v \leftarrow f_v(\mathbf{x}, \{(\mathbf{y}_w, w)\}_{w\in\mathcal{N}^-(v)}; \rho)$
  $\mathbf{y}_v \leftarrow \mathcal{O}_{c(v)}(\mathbf{x}_v)$
**return** $f_t(\mathbf{x}, \{\mathbf{y}_w\}_{w\in\mathcal{N}^-(t)})$

Algorithm $\tau(\mathcal{G}, s)$

$L \leftarrow []$
$Q \leftarrow []$
**while** $Q \neq []$ **do**
  $v \leftarrow Q$ // Topological ordering TODO

**Figure 3.1:** Interpretation of $\left(\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}\right)$ into an algorithm. $\tau$ returns the vertices in $\mathcal{G}$ in topological order starting from the source $s$.

algorithm input and the output of the sink $t_{alg}$ to be the output of the combiner. The input $\mathbf{x}_v$ to the query at vertex $v \in V_{alg} \setminus \{s_{alg}, t_{alg}\}$ is computed by an efficient (ppt.) Turing machine $f^v_{alg}$ associated with vertex $v$ in $G_{alg}$ which takes as input the labeled outputs of all predecessors $w \in \mathcal{N}_{in}(v)$ and produces an oracle query $\mathbf{x}_v$. We formally describe robust black-box DAG combiners in the following definition.

**Definition 3.1 (Robust $(n;k)$ Black-Box DAG P-Combiner)** *Given a cryptographic primitive* P *implementing a set of algorithms indexed by* $j \in [|\mathsf{P}|]$. *A robust* $(n;k)$ *black-box DAG* P*-combiner is a* $|\mathsf{P}|$*-tuple*

$$\left(\mathfrak{G}_j = \left(\mathcal{G}_j, c_j, s_j, t_j, \{f_{j,v}\}_{v\in V(\mathcal{G}_j)}\right)\right)_{j\in[|\mathsf{P}|]}$$

*where* $\mathcal{G}_j$ *is a directed acyclic graph with vertex set* $V(\mathcal{G}_j)$, $c_j : V(\mathcal{G}_j) \setminus \{s_j, t_s\} \to [n] \times [|\mathsf{P}|]$ *is a vertex-coloring with colors* $(a, b)$ *indicating that the corresponding query is made to algorithm* $a$ *of candidate oracle* $b$, *source and sink* $s_j, t_s \in V(\mathcal{G}_j)$ *satisfying* $\mathcal{N}^-(s_j) = \mathcal{N}^+(t_j) = \varnothing$ *and for simplicity the only vertices satisfying those properties and finally* $\{f_{j,v}\}_{v\in V(\mathcal{G}_j)}$ *are ppt. Turing machines. satisfying that the following tuple of oracle Turing machines is a robust* $(n;k)$ *black-box combiner.*

$$\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}} = \left(\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_1]\!], \ldots, \widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_{|\mathsf{P}|}]\!]\right)$$

*Where* $[\![\mathfrak{G}_i]\!]$ *is the interpretation according to Figure 3.1.*

A robust black-box DAG combiner algorithm is then executed by tracing the computation through the corresponding DAG $G_{alg}$, starting at the source $s_{alg}$ and ending at the sink $t_{alg}$ in a topological ordering. The interpretation of an algorithm $alg$ described by the robust black-box DAG combiner by $(\mathcal{G}_{alg}, s_{alg}, t_{alg}, l_{alg}, \{f^v_{alg}\}_{v\in V_{alg}\setminus\{s_{alg},t_{alg}\}})$ is shown in Figure 3.1.

**Example.** We introduce the notion of robust black-box DAG combiners to make the underlying structure of robust black-box combiners more explicit. To illustrate the definition we will show the structure of the DAG combiner of a robust $(2;1)$ black-box combiner **??** in Section 3.1. We label every vertices $v$ in the DAGs that is not a source or sink with $l_{alg}(v)$, i.e. their vertex labling and for every vertex add the functional description of the Turing machines $f^v_{alg}$.

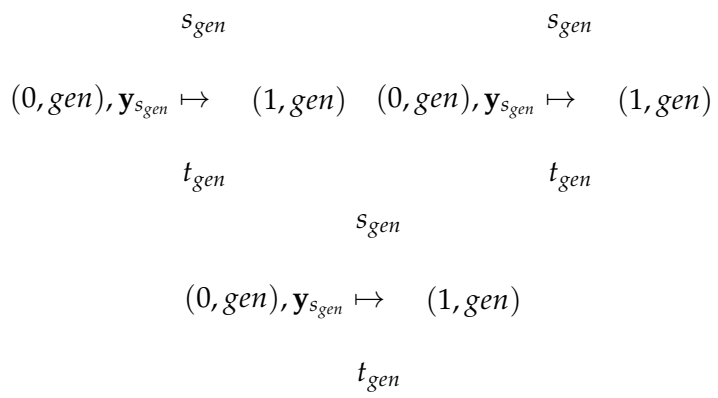



**Figure 3.2:** Drawing of the graphs $\mathcal{G}$

Note that while this is a DAG combiner implements the same robust $(2;1)$ black-box combiner as in **??**, but not the combiner that would be constructed by the constructive proof below since that is much more tractable. The DAG combiner constructed in the proof is shown in **??**.

Since later on we will focus on showing the impossibility of certain combiner constructions it's essential that the notion of combiner that we use is as strong as possible. By definition a robust black-box DAG combiner is also a robust black-box combiner. We show that the notions of robust black-box DAG combiner and robust black-box combiner are equally strong and capture the same set of constructions. We formulate this result in the following theorem.

**Theorem 3.2 (Generality of Robust $(n;k)$ Black-Box DAG Combiners)** *For every robust $(n;k)$ black-box combiner there exists a computationally equivalent robust $(n;k)$ black-box DAG combiner.*

**Intuition.** We need to show that for every robust $(n;k)$ black-box combiner $\Pi$ we can construct a robuts $(n;k)$ black-box DAG combiner $\widetilde{\Pi}$ that represents the same function if interpreted, given that $\Pi$ is efficient is also efficient. The proof works by unrolling the computation done by the combiner $\Pi$ for every oracle call made to a candidate scheme. Since we know that $\Pi$ is a black-box combiner, it runs in time polynomial in $\lambda, n, m$. Let $T \in \mathsf{poly}(\lambda, n, m)$ be an upper bound on the number of steps that the algorithm runs over all inputs $\mathbf{x}$ into the combiner. We start by constructing a graph $\mathcal{G}_{alg}$ that consists of $T$ layers of $nm$ vertices, where $n$ is the number of candidates and $m$ is the number of algorithms implemented by the primitive. On every layer, each vertex corresponds to an oracle call to a specific oracle algorithm of a candidate. We connect two vertices $v, w \in V_{alg}$ if the they are not in the same layer with an edge pointing from the vertex in the layer closer to the source to the other vertex. To define the functions $f^v_{alg}$ for $v \in V_{alg}$, a vertex in layer $k \in [T]$, we construct a Turing machine that simulates the robust black-box combiner $\Pi$ until the $k$'th oracle query. Once the $i \in [k-1]$'th oracle query is made we instead use the correct output of the correct vertex in the $i$'th layer and move on. For the $k$'th query we simply query the oracle of our vertex if the oracle is correct (and also if its is not correct, this will then simply be ignored later one during the execution).

To prove that this construction results in the same output as the robust black-box combiner does, we proceed by induction on the layer depth of the current query, claiming that the output of the correct vertex at that layer aligns with the query output of the corresponding query in the execution of the robust black-box combiner. Note that the DAG only contains $T \times n \times m$ many vertices and Turing machines $f^v_{alg}$. Since the computation at every vertex is bounded from above by the time it takes the robust black-box combiner to run and we only make one oracle call at every vertex, the result of every vertex can be computed in time at most $T$. Overall the time it takes for the robust black-box DAG combiner to run is therefore polynomial in $\lambda, n, m$. We provide a formal proof below.

**Proof (Theorem 3.2)** Let $\mathsf{P}$ be a cryptographic primitive $\Pi_{\mathsf{P}}$ with defining algorithms $(p_1, \ldots, p_{|\mathsf{P}|})$, et $\Pi_{\mathsf{P}}$ be a robust $(n;k)$ black-box $\mathsf{P}$-combiner according to Definition 2.12, and let $j \in [|\mathsf{P}|]$ be arbitrary. By Definition 2.12 there exists a polynomial $T \in \mathsf{poly}(n, \lambda, |x|)$ where $x$ is the input to $p_j$. We construct a directed acyclic graph $\mathcal{G}_j$

$$\mathcal{G}_j = ([n] \times [|\mathsf{P}|] \times [T] \cup \{s,t\}, \{\, ((a_1,b_1,t_1),(a_2,b_2,t_2)) : t_1 < t_2 \,\})$$

and connect to s,t

. We define the coloring by $c(a,b,t) = (a,b)$. We define the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ iteratively starting from the beginning of the computation $s$, we define $f_{j,v_{j,a,b,1}}$ to be the ppt Turing machine that simulates the $\Pi_{\mathsf{P}}.p_j$ until the first call to its oracle. If that oracle happens to be $(a,b)$ then we end, otherwise write 0 onto the tape. In every subsequent level $f_{j,a,b,t+1}$ simulates the combiner on and answers the queries to the respective oracle with the output of that query ($y_{j,a,b,k}$ for $k \in [t]$) by progressively simulating all previous Turing machines $f_{j,a,b,k}$ for $k \in [t]$. By checking whether after the computation we find a 0 on the tape we can trace the path of queried oracles. We can now continue the simulation of the combiner since we know exactly the state of the tape. . The combiner is necessarily efficient since for every the number of vertices in the graph is bounded from above by $nT(n,\lambda,|x|)|\mathsf{P}|$ where $T \in \mathsf{poly}(m,\lambda,|x|)$, we only make exactly one query to any oracle per vertex (which also puts a polynomial bound on the number of oracle calls), the topological ordering can be computed in polynomial time. We are only left to show that the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ are indeed ppt. To this end note that $f_{j,v}$ simulates the black-box combiner Turing machine, which is assumed to be ppt, at most once where it can answer any oracle query by looking up the corresponding output of the layer in time polynomial in $n, |\mathsf{P}|$ and $\lambda$. □

maybe inductive correctness proof but this seems to be very straight forward

## 3.2 Impossibility of Robust Black-Box VRF-Combiner

We consider the possibility of combining VRFs in a black-box fashion. Specifically we study possibilities of combining two candidates, at least one of them implementing a secure VRF. Such robust $(2;1)$ black-box combiners are the simplest combiner construction, the existence of a robust $(2;1)$ black-box combiner immediately gives rise to a robust $(n;1)$ black-box combiner while an impossibility implies that there exist no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner.

In this section we prove the impossibility of robust $(2;1)$ black-box VRF-combiners, for that we make the simplifying assumption that the secret key space of all candidates is given by $\mathcal{SK}_i = \{0,1\}^{s_i(\lambda)}$ or that at least there exists a bijection from $\mathcal{SK}_i$ to $\{0,1\}^{s_i(\lambda)}$ for $s_i \in \mathsf{poly}(\lambda)$. We state the result formally in the following theorem.

**Theorem 3.3 (Robust $(2;1)$ Black-Box VRF Combiner)** *There exists no robust $(2;1)$ black-box combiner as define in Definition 2.12 for the VRF primitive as defined in Definition 2.9.*

**Intuition.** Instead of showing the impossibility for the robust $(2;1)$ black-box combiner, we show an impossibility for robust $(2;1)$ black-box DAG combiners. The impossibility of a robust $(2;1)$ black-box VRF-combiner then follows directly from the equivalence of robust black-box combiners and robust black-box DAG combiners in Theorem 3.2.

Now suppose, for a contradiction, that there exists a robust $(2;1)$ black-box DAG VRF-combiner $\Pi_{\mathsf{VRF}}$. We construct two oracles $\mathcal{O}_1, \mathcal{O}_2$, of which at least one oracle implement a secure VRF instance, such that $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ doesn't implement a secure VRF. By assumption, we know that $\Pi_{\mathsf{VRF}}$ is a secure VRF and therefore by definition uniquely provable and perfectly correct. We show the contradiction by providing an adversary $\mathcal{A}^{\mathsf{PSPACE},\mathcal{O}_1,\mathcal{O}_2}$ that breaks the pseudorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$, i.e. wins the pseudorandomness security experiment with non-negligible advantage.

The high-level idea of the construction behind the adversary is that he iteratively filters the set of secret key candidates in the pesudorandomness security experiment using his PSPACE oracle, beginning with the entire secret key space $\mathcal{SK}$. First the adversary samples a polynomial number of evaluation points $x_1, \ldots, x_{2s(\lambda)}$ and queries the security experiments evaluation oracle $\mathcal{O}_{eval}$ on those points to obtain the corresponding image-proof pairs $(y_1, \pi_1), \ldots, (y_{2s(\lambda)}, \pi_{2s(\lambda)})$. Subsequently $\mathcal{A}$, which was given the public key $\mathsf{pk}$, computes the trace of the combiners verification under all evaluation point image-proof pair pairs $(x_1, y_1, \pi_1), \ldots, (x_{2s(\lambda)}, y_{2s(\lambda)}, \pi_{2s(\lambda)})$ to obtain traces $\tau_1, \ldots, \tau_{2s(\lambda)}$. Given a tuple $(x_i, y_i, \pi_i, \tau_i)$ for some $i \in [2s(\lambda)]$ we show that there exists an ppt. Turing machine $\mathsf{EvalSim}_{\mathsf{VRF}}$ such that $\mathsf{EvalSim}_{\mathsf{VRF}}$

simulates the evaluation of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}$ on any secret key in a way that the following properties are satisfies, let $\mathsf{sk}^*$ be the hidden secret key of the pseudorandomness security experiment.

- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes no oracle queries
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, \tau_i) = \top$ if $\mathsf{sk} = \mathsf{sk}^*$
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, \tau_i) = \bot$ for all $\mathsf{sk}$ such that $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x_i) \neq y_i$

We will first explain how the adversary can use such an algorithm to break the pseudorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$. Recall that we give the adversary against the combiner access to a PSPACE oracle, exactly because we want the combiner to derive its security from the candidates. What we have arrived at now however is that we can find an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}$ which doesn't make any oracle queries and identifies a secret key $\mathsf{sk} \in \mathcal{SK}$ only if the execution of the evaluation algorithm of the combiner returns the same image $y$ under the true secret key and the input secret key. The adversary can now use his PSPACE complete oracle to iterate over all keys and sample a random candidate key after filtering it for many tuples $(x_i, y_i, \pi_i, \tau_i)$. Now if we filter for sufficiently many keys and the size of the key space is reduced enough in every step on average we quickly end up with a polynomially bounded set of candidates at which point we can guess the correct secret key with non-negligible probability and win the security experiment. If the size of the candidate space doesn't decrease "much" then this means that a large fraction of the secret keys in the candidate set result in the same image as the true secret key. Again, after sufficiently many rounds this means that for the next uniformly random evaluation point a large fraction of the candidates still evaluates to the same image with high probability. Also in this case we can sample a random candidate key and break the pseudorandomness security experiment with non-negligible probability.

Now onto the construction of the $\mathsf{EvalSim}_{\mathsf{VRF}}$ algorithm. The algorithm works as follows: Simulate the execution of the combiners evaluation algorithm until we encounter the first oracle query. For the oracle query we check if it is repeated in the corresponding trace $\tau_i$, we will call such queries *exposed*. Those queries we can always answer with the correct output. If we find queries which are not contained in the trace then we will branch, and iterate over all possible outputs of the correct format of the query and for every assignment continue execution, we call such vertices *obscure*. We will now initialize one of the candidates, say $\mathcal{O}_1$ with a uniquely provable and perfectly correct oracle that always evaluates to 0 and verifies images if and only if they are 0, initialize the output of the other oracle $\mathcal{O}_2$ with a secure VRF. We argue that the two properties are satisfied.

difference trace and transcript

The specific construction of $\mathsf{EvalSim}_{\mathsf{VRF}}$ is illustrated in **??**. Its easy to see that the first and the third property holds. By construction we don't make

any queries to oracles and by iterating over all possible (correctly formed outputs) of the oracles for all hidden vertices and answering honestly for all visible vertices one of the executions that we enumerate corresponds exactly to the execution of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$, if that doesn't evaluate to $y$ then the algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}$ definitionally evaluates to $\perp$. For the first case we make the following observation: The only oracle answeres that we cannot answer by looking at the transcript are the hidden queries made to oracle $\mathcal{O}_2$, for any possible assingment of those values the combiner output cannot change.

The proof for this statement works by iterating through the vertices of $\mathcal{G}_{eval}$ in a fixed topological ordering until we reach a hidden vertex $v_i$ associated with oracle $\mathcal{O}_2$ for which there exist at least two distinct assignments that result in a different combiner output (over all possible previous assignments to the hidden $\mathcal{O}_2$ vertices). Then we show that we can break either of the assumed unique provability and the perfect correctness of the combiner by first turning the uniquely provable and perfectly correct oracle $\mathcal{O}_1$ into a VRF, by remapping all but a small subset of points to agree with a secure VRF. We formally treat this case in **??**. Secondly we can w.l.o.g. assume that every specific algorithm of every specific oracle is only queried on an input once, we elaborate on that in Claim 3.10. Finally we now produce two different oracles $\hat{\mathcal{O}}_1, \tilde{\mathcal{O}}_1$ such that they produce different outputs $\hat{y} \neq \tilde{y}$ but for the oracles the verification algorithm needs to be semantically equivalent. This results in the combiner either not being uniquely provable or perfectly correct.

We show the description of the full adversary $\mathcal{A}$ in Figure 3.3.

**Proof of Theorem 3.3.** Let $\Pi_{\mathsf{VRF}}$ be an arbitrary robust $(2;1)$ black-box DAG combiner for the VRF primitive. We claim that there exists a related robust black-box DAG combiner for the VRF primitive that only queries every implementation function of every candidate scheme on any input **x** at most once.

**Claim 3.4** *Let $\Pi_{\mathsf{VRF}}$ be a robust $(2;1)$ black-box DAG combiner for the VRF primitive, there exists a robust $(2;1)$ black-box DAG combiner for the VRF primitive $\widetilde{\Pi_{\mathsf{VRF}}}$ such that in its evaluation and verification algorithm DAG $\mathcal{G}_{eval}$ and $\mathcal{G}_{vfy}$ and each of the vertices $v \in V_{eval}$ and $w \in V_{vfy}$ the call to*

The idea is to slightly alter the intermediate Turing machines to check, before they make a query, whether the oracle has already been queried on that specific input. If that is the case then the oracle is queried on a fresh input but the output is ignored. Subsequent queries that would depend on that query re-simulate the execution anyway and therefore know on which value said oracle was queries, they then always refer to the value returned on

the topologically first query. Assume that this would change the combiner execution for some query, then this means that the oracle output were not deterministic and based on the internal randomness the combiner output changes. By perfect correctness both resulting distinct outputs must verify which would violate unique provability.

Additionally we show that given two VF oracles $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ such that $\mathcal{O}_{\mathbf{VRF}}$ implements a secure VRF and $\mathcal{O}_{up}$ implements a perfectly correct and uniquely provable VF we can construct a new oracle $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}})$ such that $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ implements a secure VRF and its evaluation algorithm agrees on all points $(\mathsf{sk}, x)$ with the oracle $\mathcal{O}_{up}$ where $\mathsf{sk} \in \widetilde{\mathcal{SK}}$ and $\mathsf{pk} \in \widetilde{\mathcal{PK}}$ if $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. Formally:

**Lemma 3.5** *Let $\mathcal{O}_{\mathbf{VRF}}$ be a VF oracle that implements a secure VRF in the sense of Definition 2.9, $\mathcal{O}_{up}$ be a VF oracle that is uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4 such that $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ are on the same spaces and let $\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}$ be polynomially bounded subsets of the secret and public key spaces of both oracles. There exists an oracle $\mathcal{O}_{mix}$ such that the following properties are satisfied:*

1. *The evaluation algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all secret keys $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}, \mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot) = \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot)$$

2. *The verification algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all public keys $\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}, \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot) = \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot)$$

3. *The oracle $\mathcal{O}_{mix}$ implements a secure VRF in the sense of Definition 2.9.*

Note first that it is not sufficient to simply change all of the mappings of the evaluation algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{SK}}$ to the output of $\mathcal{O}_{up}$ and the mappings of the verification algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{PK}}$ to the output of $\mathcal{O}_{up}$. Such a construction would indeed not necessarily be uniquely provable or perfectly correct since for one we might have not guaranteed that the verification of a newly mapped output succeeds or the verification of a previously mapped output rejects and also we need to consider that if two secret keys have non-zero probability to be sampled with the same public key, then they both need to induce the same mapping $\mathcal{X} \to \mathcal{Y} \times \mathcal{P}$, also this holds for the public key perspective if we change the mappings of the verification algorithm.

**Proof (Lemma 3.5)** We begin by introducing a two relations $\sim_{\mathcal{SK}} \subseteq \mathcal{SK}^2$ and $\sim_{\mathcal{PK}} \subseteq \mathcal{PK}^2$. Intuitively, these relations capture whether two secret or public keys have a counterpart which could have been sampled with both keys. Formally we define the relations $\sim_{\mathcal{SK}}, \sim_{\mathcal{PK}}$ as follows:

$$\mathsf{sk}_1 \sim_{\mathcal{SK}} \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\begin{bmatrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{bmatrix} > 0 \tag{3.1}$$

$$\mathsf{pk}_1 \sim_{\mathcal{PK}} \mathsf{pk}_2 \iff \exists \mathsf{sk} \in \mathcal{SK}, \Pr\begin{bmatrix}(\mathsf{sk}, \mathsf{pk}_1) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}, \mathsf{pk}_2) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{bmatrix} > 0 \tag{3.2}$$

We now consider the transitive closure of those relations $\sim^*_{\mathcal{SK}}, \sim^*_{\mathcal{PK}}$, formally defined by $\sim^* = \bigcup_{k\in\mathbb{N}} \sim^k$. The resulting relations $\sim^*_{\mathcal{SK}}, \sim^*_{\mathcal{PK}}$ are equivalence relations. Given an element $\mathsf{sk}$ we denote its equivalence class under $\sim^*_{\mathcal{SK}}$ with $[\mathsf{sk}]_{\sim^*_{\mathcal{SK}}} = \{\mathsf{sk}' \in \mathcal{SK} : \mathsf{sk}' \sim^*_{\mathcal{SK}} \mathsf{sk}\}$, similarly for an element $\mathsf{pk}$ and the relation $\sim^*_{\mathcal{PK}}$. We claim that keys related with the corresponding relation $\sim^*$ must induce the same mapping on their function. We formalize this observation in the following claim.

**Claim 3.6** *The relations $\sim^*_{\mathcal{SK}}$ and $\sim^*_{\mathcal{PK}}$ defined in* (3.1) *and* (3.2) *respectively are equivalence relations, i.e. reflexive, symmetric and transitive.*

The relation $\sim_{\mathcal{SK}}, \sim_{\mathcal{PK}}$ are both reflexive and symmetric which is easy to see from their definition. Further the transitive closure of a reflexive and symmetric relation is always an equivalence relation.

Additionally, for secret and public keys we define the function $R_{\mathcal{SK}}$ and $R_{\mathcal{PK}}$ respectively as follows. $R$ maps a key to all counterparts with are sampled with that key with non-negligible probability.

$$R_{\mathcal{PK}}(\mathsf{pk}) = \left\{\mathsf{sk} \in \mathcal{SK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.3}$$

$$R_{\mathcal{SK}}(\mathsf{sk}) = \left\{\mathsf{pk} \in \mathcal{PK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.4}$$

Next we define the notion of the closure of a set. Given a subset of secret $\widetilde{\mathcal{SK}}$ or public keys $\widetilde{\mathcal{PK}}$ we define its closure $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ and $\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})$ which are just the union over the equivalence classes of all keys in the subsets. We formally define the functions as follows:

$$\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}) = \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}} \tag{3.5}$$

$$\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}}) = \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}} \tag{3.6}$$

Observe that the secret key closure $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ for a subset $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ is the set of secret keys that need to be remapped if we want to remap all of

the evaluations on the secret keys $\widetilde{\mathcal{SK}}$ to preserve unique provability and correctness. The same holds for the public key closure. Now we define the closure for subsets $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ at the same time.

$$\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) = \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}) \cup \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}}) \tag{3.7}$$

$$\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) = \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}}) \cup \bigcup_{\widetilde{\mathsf{sk}} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})} R_{\mathcal{SK}}(\widetilde{\mathsf{sk}}) \tag{3.8}$$

We now constuct an oracle $\widetilde{\mathcal{O}}_{up}$ that interpolates between $\mathcal{O}_{up}$ and $\mathcal{O}_{\mathbf{VRF}}$. To be precise we define all of the algorithms of $\widetilde{\mathcal{O}}_{up}$: The key generation algorithm is defined to be the function for key generation of $\mathcal{O}_{\mathbf{VRF}}$ while the function for evaluation and verification are defined as follows.

$$\widetilde{\mathcal{O}}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = \begin{cases} \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{if } \mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{otherwise} \end{cases} \tag{3.9}$$

$$\widetilde{\mathcal{O}}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = \begin{cases} \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{if } \mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{otherwise} \end{cases} \tag{3.10}$$

Note that the properties 1. and 2. of Lemma 3.5 hold by definition of the oracle since by reflexivity of $\sim^*_{\mathcal{SK}}$ and $\sim^*_{\mathcal{PK}}$ we have $\widetilde{\mathcal{SK}} \subseteq \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ and $\widetilde{\mathcal{PK}} \subseteq \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. We show that 3. holds by showing the $\widetilde{\mathcal{O}}_{up}$ is uniquely provable and perfectly correct and pseudorandom seperately.

**Claim 3.7** *Let $\mathcal{O}_{up}, \mathcal{O}_{vrf}$ be oracles implementing a uniquely provable VF in the sense of Definition 2.5 and a VRF in the sense of Definition 2.9 respectively. The oracle $\mathcal{O}_{mix}$ constructed from $\mathcal{O}_{up}$ and $\mathcal{O}_{vrf}$ is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5.*

Let $\lambda \in \mathbb{N}$ be arbitrary. For perfect correctness let $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$, let $x \in \{0,1\}^\lambda$ be arbitrary and let $(y, \pi) \leftarrow \mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$. We show that $\mathsf{sk} \in \mathcal{SK}_{map}$ if and only if $\mathsf{pk} \in \mathcal{PK}_{map}$, following that, if $\mathsf{sk} \in \mathcal{SK}_{map}$ then $(y, \pi)$ verifies by perfect correctness of $\mathcal{O}_{up}$ and if $\mathsf{sk} \notin \mathcal{SK}_{map}$ then $(y, \pi)$ verifies by perfect correctness of $\mathcal{O}_{vrf}$.

We show only $\mathsf{sk} \in \mathcal{SK}_{map} \implies \mathsf{pk} \in \mathcal{PK}_{map}$, the other direction follows from a symmetric argument. Assume $\mathsf{sk} \in \mathcal{SK}_{map}$, by definition of $\mathcal{SK}_{map}$ we distinguish the cases $\mathsf{sk} \in \mathsf{cl}(\widetilde{\mathcal{SK}})$ and $\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}})$. If $\mathsf{sk} \in \mathsf{cl}(\widetilde{\mathcal{SK}})$ then together with $\mathsf{pk} \in R_{\mathcal{PK}}(\mathsf{sk})$ we get $\mathsf{pk} \in \bigcup_{\widetilde{\mathsf{sk}} \in \mathsf{cl}(\widetilde{\mathcal{SK}})} R_{\mathcal{SK}}(\widetilde{\mathsf{sk}})$ and therefore $\mathsf{pk} \in \mathcal{PK}_{map}$. On the other hand if $\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}})$ then for some $\widetilde{\mathsf{pk}} \in \mathsf{cl}(\widetilde{\mathcal{PK}})$ we have $\mathsf{sk} \in R_{\mathcal{PK}}(\widetilde{\mathsf{pk}})$ but since $\widetilde{\mathsf{pk}} \sim_{\mathcal{PK}} \mathsf{pk}$ and therefore in particular $\widetilde{\mathsf{pk}} \sim^*_{\mathcal{PK}} \mathsf{pk}$ we have $\mathsf{pk} \in \mathsf{cl}(\widetilde{\mathsf{pk}})$, implying $\mathsf{pk} \in \mathcal{PK}_{map}$.

For unique provability, suppose for a contradiction, that there exist two distinct images $y_0 \neq y_1$ with corresponding proofs $\pi_0, \pi_1$ such that $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_0, \pi_0) = \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1$. Now, if $\mathsf{pk} \in \mathcal{PK}_{map}$ then we break the assumed unique provability of $\mathcal{O}_{up}$ and if $\mathsf{pk} \notin \mathcal{PK}_{map}$ then we break the assumed unique provability of $\mathcal{O}_{vrf}$. We conclude that the constructed oracle $\mathcal{O}_{mix}$ must indeed be perfectly correct and uniquely provable.

Next we show that if $\mathcal{O}_{\mathbf{VRF}}$ is pseudorandom then $\mathcal{O}_{mix}$ is also pseudorandom. We formulate this statement precisely in the following claim.

**Claim 3.8** *Let $\mathcal{O}_{up}, \mathcal{O}_{vrf}$ be oracles implementing a uniquely provable VF in the sense of Definition 2.5 and a VRF in the sense of Definition 2.9 respectively that is $(t, \varepsilon)$-pseudorandom. Let $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}, \widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ be polynomially bounded subsets of the secret and public key space respectively, i.e. $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. Then $\widetilde{\mathcal{O}}_{mix}^{\widetilde{\mathcal{SK}},\widetilde{\mathcal{PK}},\mathcal{O}_{up},\mathcal{O}_{vrf}}$ which is constructed from $\mathcal{O}_{up}$ and $\mathcal{O}_{vrf}$ using the above procedure is $(t, \varepsilon)$-pseudorandom for $\varepsilon_{mix}^{prf} = \varepsilon_{vrf}^{prf} + \mu_1(\lambda) + \mu_2(\lambda)$ where both $\mu_1(\lambda)$ and $\mu_2(\lambda)$ are negligible functions depending on the size of the closure of $\widetilde{\mathcal{SK}}$ and $\widetilde{\mathcal{PK}}$ respectively.*

Assume, towards a contradiction, that $\mathcal{O}_{mix}$ is not pseudorandom, i.e. that there exists an efficient (ppt.) adversary $\mathcal{A}$ such that $\mathcal{A}$ wins the pseudorandomness experiment against $\mathcal{O}_{mix}$ with non-negligible advantage. We now show that the pseudorandomness game against $\mathcal{O}_{mix}$ and $\mathcal{O}_{\mathbf{VRF}}$ are statistically close, i.e. we bound the probability that the hidden secret key $\mathsf{sk} \in \mathcal{SK}_{map}$ and apply the difference lemma. We define the following games $H_1(\lambda, \mathcal{A}), H_2(\lambda, \mathcal{A})$ and $H_3(\lambda, \mathcal{A})$ corresponding to the pseudorandomness security experiment for hidden bit $b = 0$ against $\mathcal{O}_{\mathbf{VRF}}$, the pseudorandomness security experiment for hidden bit $b = 1$ against $\mathcal{O}_{\mathbf{VRF}}$ and the pseudorandomness security experiment for hidden bit $b = 1$ against $\mathcal{O}_{mix}$ respectively as defined in Figure 2.1.

We use $H_i^{prf}(\lambda, \mathcal{A}) = 1$ to denote the event that adversary $\mathcal{A}$ wins the game $H_i$ for $i \in [3]$.

define games of the hybrid formally

$$\mathbf{Adv}^{\text{prf}}_{\mathsf{VRF}}(\mathcal{A}, \mathcal{O}_{mix}) = \left|\Pr\left[H_0^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_1^{prf}(\lambda, \mathcal{A}) = 1\right]\right| \tag{3.11}$$

$$= \Big| \Pr\left[H_0^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_2^{prf}(\lambda, \mathcal{A}) = 1\right] \tag{3.12}$$

$$+ \Pr\left[H_2^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_1^{prf}(\lambda, \mathcal{A}) = 1\right]\Big| \tag{3.13}$$

$$\leq \left|\Pr\left[H_0^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_2^{prf}(\lambda, \mathcal{A}) = 1\right]\right| \tag{3.14}$$

$$+ \left|\Pr\left[H_2^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_1^{prf}(\lambda, \mathcal{A}) = 1\right]\right| \tag{3.15}$$

$$\leq \varepsilon^{prf}_{VRF,mix} + \left|\Pr\left[H_2^{prf}(\lambda, \mathcal{A}) = 1\right] - \Pr\left[H_1^{prf}(\lambda, \mathcal{A}) = 1\right]\right| \tag{3.16}$$

Let $\mathcal{E}$ be the event that $\mathsf{sk} \in \mathcal{SK}_{map}$ for $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$. Observe that the games $H_2, H_1$ are identical if the event $\mathcal{E}$ doesn't occur. The difference lemma is a classical result in cryptography and states that the difference in probability of an event happening can be bounded from above by the probability of a "bad" event happening conditioned on which the previous two events have the same probability of happening. For completeness we state the result but defer the proof to the appendix.

**Lemma 3.9 (Difference Lemma)** *let $W_0, W_1$ and $B$ be events satisfying:*

$$\Pr[W_0 \mid \neg B] = \Pr[W_1 \mid \neg B]$$

*Then we can impose the following bound:*

$$|\Pr[W_0] - \Pr[W_1]| \leq \Pr[B]$$

By the above observation we can therefore bound the difference in probability over the two games by the probability of event $\mathcal{E}$ occuring. Applying a union bound and the definition of the joint closure we then arrive at the following bound of the adversary $\mathcal{A}$'s advantage.

$$\mathbf{Adv}^{\text{prf}}_{\mathsf{VRF}}(\mathcal{A}, \mathcal{O}_{mix}) \leq \varepsilon^{prf}_{VRF,mix} + \Pr\left[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})\right] \tag{3.17}$$

$$\leq \varepsilon^{prf}_{VRF,mix} + \Pr\left[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})\right] \tag{3.18}$$

$$+ \Pr\left[\mathsf{sk} \in \bigcup_{\widehat{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widehat{\mathsf{pk}})\right] \tag{3.19}$$

$$\leq \varepsilon^{prf}_{\mathbf{VRF}} + |\widetilde{\mathcal{SK}}|\varepsilon^{prf}_{\mathbf{VRF}} + |\widetilde{\mathcal{PK}}|\varepsilon^{prf}_{\mathbf{VRF}} \tag{3.20}$$

We bound the probabilities and arrive at the final bound we show that if the probability were non-negligible an adversary would win the pseudorandomness security experiment against $\mathcal{O}_{\mathbf{VRF}}$ with non-negligible advantage,

which is a contradiction since we assume $\mathcal{O}_{\mathbf{VRF}}$ to be pseudorandom. For the subsequent proofs keep in mind that the probabilities are over the randomly sampled secret key. We consider both probabilities seperately,

For the bound on $\Pr[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})]$ note that for any value of $x$ the set of possible images is given by $\{y \leftarrow_{\$} \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\widetilde{\mathsf{sk}}, x) : \widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}\}$ which is bounded in cardinality from above by $\widetilde{\mathcal{SK}}$. Recall that $|\widetilde{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$. Conditioned on the event that $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ there exists an adversary $\mathcal{A}$ with advantage at least $1/|\widetilde{\mathcal{SK}}|$. Hence we can bound $\Pr[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})]$ from above by $\varepsilon^{prf}_{VRF,mix}|\widetilde{\mathcal{SK}}|$.

For the bound on $\Pr[\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}})]$ by a similar argument the mappings also correspond to elements of $\widetilde{\mathcal{PK}}$. Again, following this line of argument we condition on the event that $\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}})$ and find an adversary with advantage bounded from below by $1/|\widetilde{\mathcal{PK}}|$. We obtain the bound on the probability of $\varepsilon^{prf}_{\mathbf{VRF}}|\widetilde{\mathcal{PK}}|$. □

Using this result we claim that for a fixed oracle $\mathcal{O}_i$ and any subset of queries made during the evaluation algorithm which are not reflected in the trace of the verification execution, we call such queries *hidden*, the output of the evaluation algorithm doesn't change for all possible outputs of the oracles. That is, if we execute the evaluation algorithm and replace every oracle query to an algorithm of $\mathcal{O}_i$ with the output in the transcript or any other random output in the support if it doesnt occur in the transcript we arrive at the same value for the right secret key $\mathsf{sk}$.

**Claim 3.10** *Let $\mathcal{O}_1, \mathcal{O}_2$ be uniquely provable and perfectly correct VF oracles and let $S$ be a set of hidden vertices in $G_{eval}$ with queries made to only one oracle. Then they are jointly irrelevant to the output of the combiners evaluation algorithm. Specifically we can simulate the output of the individual oracle queries by choosing arbitrary elements in the support of the oracles algorithm.*

**Proof of Claim 3.10** Assume this were not the case, we follow the topological ordering and initialize the oracle responses to those queries with arbitrary outputs in the support. If at one point the output of the combiner changes then simply remap the corresponding oracle and make the other oracle a VRF. Subsequently the combiner needs to be uniquely provable, perfectly correct and pseudorandom. By perfect correctness we have

$$\Pi^{\hat{\mathcal{O}}_1, \mathcal{O}_2^{\mathbf{VRF}}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \hat{y}, \hat{\pi}) = 1 \tag{3.21}$$

$$\Pi^{\tilde{\mathcal{O}}_1, \mathcal{O}_2^{\mathbf{VRF}}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \tilde{y}, \tilde{\pi}) = 1 \tag{3.22}$$

And by unique provability and $\hat{y} \neq \tilde{y}$ we have that

$$\Pi_{\mathsf{VRF}}^{\hat{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \tilde{y}, \tilde{\pi}) = 0 \tag{3.23}$$

$$\Pi_{\mathsf{VRF}}^{\tilde{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \hat{y}, \hat{\pi}) = 0 \tag{3.24}$$

Now since the oracle $\tilde{\mathcal{O}}_1$ and $\hat{\mathcal{O}}_1$ only differ in the evaluations not covered by the verification algorithm the execution of the algorithm will be identical, given the input is the same. This fundamentally contradicts (3.22) and (**??**), we conclude that the resulting combiner is either not perfectly correct or uniquely provable.

We define the evaluation simulation $\mathsf{EvalSim}_{\mathsf{VRF}}$ that simulates the evaluation without invoking any oracle queries. The simulation will simply traverse the evaluation algorithm of the DAG combiner. For any oracle query it checks the corresponding transcript $\tau_i$. If the query is either to oracle $\mathcal{O}_1$ we can honestly answer it with 0 or in the transcript, then we can answer it hoenstly as well. If the query is *obscure* then we branch once for every possible assignment of the oracle response which fits the given format, e.g. for a query to eval we branch for every answer $(y, \pi) \in \mathcal{Y} \times \mathcal{P}$. The pseudocode for $\mathsf{EvalSim}_{\mathsf{VRF}}$ is shown in **??**. In the following we show that this algorithm satisfies a few properties.

**Claim 3.11** *Let* $\mathsf{EvalSim}_{\mathsf{VRF}}$ *be the algorithm as described above and in* **??** *taking as input a tuple* $(\mathsf{sk}, x, y, \pi, \tau)$ *containing a secret key, evaluation point, image, proof and a trace of the combiners verification algorithm on* $(\mathsf{pk}, x, y, \pi)$ *where* $\mathsf{pk}$ *is the true public key. Then* $\mathsf{EvalSim}_{\mathsf{VRF}}$ *satisfies the following properties.*

1. $\mathsf{EvalSim}_{\mathsf{VRF}}$ *makes no oracle queries*
2. $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \top$ *if* $\mathsf{sk} = \mathsf{sk}^*$
3. $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \bot$ *if* $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x) \neq y$

Informally this means that the evaluation simulation will return with $\top$ on a *non-empty* subset of secret keys which agrees with the true secret key $\mathsf{sk}^*$ on the evaluation of $x$.

The first property holds by construction of $\mathsf{EvalSim}_{\mathsf{VRF}}$ the third property also holds by construction and the fact that if the evaluation of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x) \neq y$ then there exists an assignment of the obscure queries such that every oracle query is answered exactly the same as in the execution of the combiner, hence the combiner outputs $\bot$. If $\mathsf{sk} = \mathsf{sk}^*$ we need to argue that for every format adhering assignment of the oracle responses to the obscure queries, the output doesn't change. Assume towards a contradiction for two assignments of the obscure queries the output of the combiner changes from $y_0$ to $y_1 \neq y_0$. Now we can apply Lemma 3.5 to construct a secure VRF from $\mathcal{O}_1$ which agrees with the old candidate on all points queried in evaluation

and verification, and reassign obscure queries. By perfect correctness both outputs $y_0, y_1$ must verify in their respective world, however on all oracle queries done by the verification combiner in the two worlds the queries agree exactly (since the queries are obscure) and we break unique provability.

Using this simulator $\mathsf{EvalSim}_{\mathsf{VRF}}$ we can describe the set of iteratively filtered candidate secret keys as follows. Let $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[2s(\lambda)]}$ be a set of $2s(\lambda)$ tuples containing a uniformly random evaluation point $x_i \in \{0,1\}^\lambda$, the evaluation of $x_i$ resulting in the tuple $(y_i, \pi_i)$ and the transcript of the combiners verification algorithm on $(x_i, y_i, \pi_i)$. The set of candidate keys is defined as:

$$\mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i) := \{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i)\}$$

We claim that for any secret key $\mathsf{sk}_{cand} \in \mathcal{SK}_{cand}$, the probability that the secret key evaluates to the same output as the evaluation algorithm of the combiner does on a uniformly random value $x \in \{0,1\}^\lambda$ is overwhelming. We state this result formally in the following claim.

**Claim 3.12** *Let $\mathcal{SK}_{cand}$ be defined as above and let $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[2s(\lambda)]}$ then we have that for any secret key $\mathsf{sk}^*$ sampled uniformly at random from the intersection of all filtered candidate sets, i.e. sampled from*

$$\mathsf{sk}^* \leftarrow_\$ \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)$$

*The probability that for a uniformly random challenge $x^* \leftarrow_\$ \{0,1\}^\lambda \setminus \{x_1, \ldots, x_{2s(\lambda)}\}$ the black-box combiner instanciated with the correct oracles evaluates to the same image as for the true secret key is bounded from below by*

$$\Pr\left[\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}^*, x) = \Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}, x)\right] \geq \frac{1}{p(\lambda)}$$

*For $p$ a polynomial in $\lambda$.*

**Proof (Claim 3.12)** For a secret key $\mathsf{sk} \in \mathcal{SK}$ we define the number of secret keys $\mathsf{sk} \in \mathcal{SK}$ for which the evaluation algorithm under $\mathsf{sk}$ on $x$ agrees with the evaluation algorithm under $\mathsf{sk}^*$ on $x$.

$$\mathcal{X}(\mathsf{sk}) = \left|\left\{x \in \{0,1\}^\lambda : \Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}, x) = \Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*, x)\right\}\right|$$

We say that a secret key is *unfavorable* if the probability of a uniformly random evaluation point differing in the output is at least $1/2$ and let $\mathcal{SK}_{unfav} \subseteq \mathcal{SK}$ be the set of unfavorable keys. Since we sample the evaluation points $x$ uniformly at random from $\{0,1\}^\lambda$ the probability of the key surviving the filtering step is given by the following:

$$\Pr[\mathsf{sk} \in \mathcal{SK}_{cand}(x, y, \pi, \tau)] = \frac{\mathcal{X}(\mathsf{sk})}{2^\lambda}$$

Since the $2s(\lambda)$ evaluation points $x$ are furthermore sampled i.i.d. we can bound the probability the a fixed secret key $\mathsf{sk} \in \mathcal{SK}$ is in the intersection of all $2s(\lambda)$ tuples by the following expression:

$$\Pr\left[\mathsf{sk} \in \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right] = \left(\frac{\mathcal{X}(\mathsf{sk})}{2^\lambda}\right)^{2s(\lambda)}$$

We can bound the expected number of unfavorable secret keys that survive $2s(\lambda)$ rounds of secret key filtering over the randomness of $x_1, \ldots, x_{2s(\lambda)}$ by applying the union bound:

$$\mathbb{E}\left[\left|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right|\right] \tag{3.25}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \Pr\left[\mathsf{sk}_{unfav} \in \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right] \tag{3.26}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[2s(\lambda)]} \Pr\left[\mathsf{sk}_{unfav} \in \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right] \tag{3.27}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[2s(\lambda)]} \frac{\mathcal{X}(\mathsf{sk})}{2^\lambda} \tag{3.28}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[2s(\lambda)]} \frac{1}{2} \tag{3.29}$$

$$\leq \frac{2^\lambda}{2^{2s(\lambda)}} \tag{3.30}$$

Which for general polynomials $s(\lambda) > \lambda$ is negligibly small. We can now use a simple Markov bound to show that the probability of a bad key surviving the filtering is negligibly small. Note that the expected number of surviving unfavorable secret keys is by definition non-negative and therefore we have that the probability of all keys filtered is given by the following expression:

add assumption?

$$\Pr\left[\left|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{card}(x_i, y_i, \pi_i, \tau_i)\right| \geq 1\right] \leq \frac{2^\lambda}{2^{2s(\lambda)}}$$

To bound the final probability simply condition on the event that no bad key survives. Let $\mathcal{E}$ be the event that $|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)| = 0$ and let $\mathcal{F}$ be the event that $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}^*, x^*) = \Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}, x^*)$. Note that if $\neg\mathcal{E}$ holds we know that only *favorable* keys survive and therefore

the probability of the same image is at least $1/2$.

$$\begin{aligned}
\Pr[\mathcal{F}] &= 1 - \Pr[\neg\mathcal{F} \mid \mathcal{E}]\Pr[\mathcal{E}] - \Pr[\neg\mathcal{F} \mid \neg\mathcal{E}]\Pr[\neg\mathcal{E}] && (3.31)\\
&\geq 1 - \Pr[\neg\mathcal{F} \mid \mathcal{E}] - \Pr[\neg\mathcal{E}] && (3.32)\\
&\geq 1 - \tfrac{1}{2} - \tfrac{2^\lambda}{2^{2s(\lambda)}} && (3.33)\\
&\geq \tfrac{1}{2} - \tfrac{2^\lambda}{2^{2s(\lambda)}} && (3.34)
\end{aligned}$$

Which concludes this proof. □

Now we define the following two languages $\mathcal{L}_{cand}, \mathcal{L}_{size}$ as follows.

$$\mathcal{L}_{cand} := \left\{ \left(\rho, i, b, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[2s(\lambda)]}\right) : \mathrm{rk}_\rho\left(\bigcap_{j\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right)_i = b \right\} \tag{3.35}$$

$$\mathcal{L}_{size} := \left\{ \left(m, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[2s(\lambda)]}\right) : \left|\bigcap_{j\in[2s(\lambda)]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right| \leq m \right\} \tag{3.36}$$

We show that $\mathcal{L}_{cand}$ and $\mathcal{L}_{size}$ are in PSPACE and can therefore be decided by the PSPACE complete oracle. The adversary will first use binary search over $m$ to recover the size of the candidate set $\mathcal{SK}_{cand}$ and then pass use the language $\mathcal{L}_{cand}$ to recover a uniformly random candidate secret key $\mathsf{sk}_{cand}$. Formally stated in the following two claims.

**Claim 3.13** *$\mathcal{L}_{cand} \in$ PSPACE and $\mathcal{L}_{size} \in$ PSPACE.*

To proof Claim 3.13 we need to provide Turing machines $\mathcal{M}_{cand}, \mathcal{M}_{size}$ that decide whether an arbitrary element is member of the language $\mathcal{L}_{cand}$ and $\mathcal{L}_{size}$ respectively such that space used by $\mathcal{M}_{cand}$ and $\mathcal{M}_{size}$ is bounded from above by functions polynomial in their input size. $\mathcal{M}_{cand}$ iterates over the keys in canonical ordering, for every secret key $\mathsf{sk} \in \mathcal{SK}$ iterate over all $i \in [2s(\lambda)]$ and for every $i$ verify whether $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top$. If that holds for every $i$ then increment a counter. If we would increment the counter but the value of the counter is $\rho$ then check the $i$'th bit of the candidate is equal to $b$.

The algorithm $\mathcal{M}_{size}$ requires works similarly. We iterate over all secret keys $\mathsf{sk} \in \mathcal{SK}$ and keep a counter to count the number of secret keys that pass every filter. For every secret key candidate $\mathsf{sk}$ we iterate over all $i \in [2s(\lambda)]$ and increment the counter if for all values $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top$. The algorithm only needs space for the counter in $\mathcal{O}(s(\lambda))$ and the execution of $\mathsf{EvalSim}_(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i)$ which is also polynomial. This concludes the proof of Claim 3.13.

We can now describe the distinguisher $\mathcal{D}$ for the pseudorandomness game as follows: $\mathcal{D}$ first samples $2s(\lambda)$ evaluation points $x_i$ uniformly at random from $\{0,1\}^\lambda$, then he uses the evaluation oracle of the pseudorandomness security experiment $\mathcal{O}_{eval}$ to evaluate the points and obtain $(y_1, \pi_1), \ldots, (y_{2s(\lambda)}, \pi_{2s(\lambda)})$. Subsequently he computes the trace of the combiners verification algorithm $\tau_i$ on $(x_i, y_i, \pi_i)$. Using binary search and at most $2s(\lambda)$ many queries to its PSPACE complete oracle it can recover the size of the set of candidate secret keys $\mathcal{SK}_{cand}$ using the language $\mathcal{L}_{size}$. He then samples a uniformly random number $\rho \in [|\mathcal{SK}_{cand}|]$ then then bit-for-bit, again using $2s(\lambda)$ many queries recovers a the $\rho$'th candidate key in canonical ordering from $\mathcal{SK}_{cand}^{\{(x_i,y_i,\pi_i,\tau_i)\}_{i\in 2s(\lambda)}}$. He then samples a uniformly random challenge $x^* \leftarrow_\$ \{0,1\}$, checking that $x^* \neq x_i$ and returns the secret key candidate $\mathsf{sk}_{cand}$ in the state and $x^*$.

In the second stage then then gets $\mathsf{sk}_{cand}, x^*$ and $y^*$. He simply recomputes $y' \leftarrow \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x)$ and returns 0 if $y^* = y'$ and 1 otherwise. We illustrate the distinguisher in Figure 3.3.

Finally, using what we proved before, we show that the distinguisher $\mathcal{D}^{\mathsf{PSPACE},\mathcal{O}_1,\mathcal{O}_2}$ has non-negligible advantage in the pseudorandomness game against $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$. We state this result formally in the final claim.

**Claim 3.14** *The advantage of the adversary $(\mathcal{D}_1^{\mathsf{PSPACE},\mathcal{O}_1,\mathcal{O}_2}, \mathcal{D}_2^{\mathsf{PSPACE},\mathcal{O}_1,\mathcal{O}_2})$ in the pseudorandomness game against the combiner $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ is overwhelming, i.e.*

$$\mathbf{Adv}_{\mathsf{VRF}}^{\mathrm{prf}}(\mathcal{A}, \lambda) \geq 1 - 3\varepsilon_{VRF,mix}^{prf}$$

While robust $(2;1)$ black-box combiners are generally sufficient to construct robust $(n;1)$ black-box combiners, non-existence of a robust $(2;1)$ black-box combiner only allows one to deduce the impossibility of a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner. This holds since we can easily construct a robust $(2;1)$ black-box combiner from a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner by running the robust $(n;\lfloor n/2 \rfloor)$ black-box combiner and for every query made to the first $\lfloor n/2 \rfloor$ oracle candidates we answer with the first oracle candidate given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ and for the upper $n - \lfloor n/2 \rfloor$ candidate we simply query the other oracle. If at least one oracle given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ implements a secure VRF, then at least $\lfloor n/2 \rfloor$ candidates correspond to a VRF oracle. We can derive the following corollary.

change title to lemma maybe

**Lemma 3.15 (Robust $(n;\lfloor n/2 \rfloor)$ Black-Box VRF Combiner)** *There exists no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive defined in Definition 2.9 for all $n \in \mathbb{N}^+$.*

Extending this result to an impossibility of robust $(n; n-1)$ seems difficult. We elaborate on the reason why the proof for **??** cannot easily be extended

Distinguisher $\mathcal{D}_1^{\mathsf{PSPACE},\mathcal{O}_{eval},\mathcal{O}_1,\mathcal{O}_2}(\mathsf{pk})$

// Obtain $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[2s(\lambda)]}$:
$\mathcal{X} \leftarrow \varnothing$
**for** $i \in [2s(\lambda)]$ **do**
  $x_i \leftarrow_{\$} \{0,1\}^\lambda$
  $\mathcal{X} \leftarrow \mathcal{X} \cup \{x_i\}$
  $(y_i, \pi_i) \leftarrow \mathcal{O}_{eval}(x_i)$
  $\tau_i \leftarrow \mathsf{trace}(\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x_i, y_i, \pi_i))$
// Obtain $|\mathcal{SK}_{cand}|$ using binary search:
$\rho \leftarrow_{\$} [k]$
// Obtain $\mathsf{sk}_{cand} \in \mathcal{SK}_{cand}$:
$\mathsf{sk}_{cand} \leftarrow \varepsilon$
**for** $i \in [s(\lambda)]$ **do**
  $b \leftarrow \mathsf{PSPACE}((0, \rho, i, \{x_i, y_i, \pi_i, \tau_i\}_{i\in[2s(\lambda)]}) \in \mathcal{L}_{cand})$
  **if** $b = \top$ **then**
    $\mathsf{sk}_{cand} \leftarrow \mathsf{sk}_{cand} \| 0$
  **else if** $b = \bot$ **then**
    $\mathsf{sk}_{cand} \leftarrow \mathsf{sk}_{cand} \| 1$
$x^* \leftarrow_{\$} \{0,1\}^\lambda \setminus \mathcal{X}$
**return** $(\mathsf{sk}_{cand}, x^*)$

Distringuisher $\mathcal{D}_2^{\mathsf{PSPACE},\mathcal{O}_{eval},\mathcal{O}_1,\mathcal{O}_2}(\mathsf{pk}, \mathsf{sk}_{cand}, x^*, y^*)$

$y' \leftarrow \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x^*)$
**return** $y' = y^*$

**Figure 3.3:** Distinguisher.

to general robust $(n; n-1)$ black-box VRF-combiners. We still believe that robust $(n; n-1)$ black-box VRF-combiners are not possible and therefore pose the following conjecture. We dedicate a brief paragraph in the appendix to show why the proof for result Theorem 3.17 cannot easily be extended to cover this more general result. Briefly, the issue is that the hidden queries, which we show are candidate-wise jointly irrelevant to the combiner output are not jointly irrelevant over multiple candidates if we increase the number of candidates. This means that we cannot simply exhaust every possible output of the hidden oracle queries and guarantee that for the correct secret key the output is always $y$.

**Conjecture 3.16** *There exists no robust $(n; n-1)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive as defined in Definition 2.9 for all natural numbers $n > 1$.*

Instead of extending the impossibility of robust $(2; 1)$ black-box VRF-combiners to an impossibility of more general robust $(n; n-1)$ black-box VRF-combiners, we show that the impossibility still holds if we lower the requirements on the combiner. Specifically we show that even if we only require the combiner to be weakly pseudorandom or unpredictable and statistically correct, there still exists no robust $(2; 1)$ black-box VRF-combiner.

**Theorem 3.17 (Weak Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VRF Combiners)** *There exist no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 that implements a VF in Definition 2.3 that is statistically correct in the sense of Definition 2.4, uniquely provable in the sense of Definition 2.5 and weakly pseudorandom in the sense of Definition 2.7, even if at least $n-1$ candidates implement a secure VRF in the sense of Definition 2.9.*

**Intuition.** The proof works similar to the proof for Theorem 3.3, we note that the current adversary already has the capability to break the weak pseudorandomness of the combiner. Additionally, even if the combiner is only required to satisfy statistical correctness, we argue that over a random secret key $\mathsf{sk}$ and a polynomial number of uniformly random evaluation points $x$ we can still expect the combiner to be correct with high probability. The proof works by additionally conditioning on the probability that the combiner is correct on all evaluations $x$ sampled during the pseudorandomness game.

**Proof (Theorem 3.17)** Again we fix a robust $(2; 1)$ black-box VRF-combiner $\Pi_{\mathsf{VRF}}$ that is only $\varepsilon$-correct for some $\varepsilon \in \mathsf{negl}(\lambda)$. Let $\mathcal{E}$ be the event that under the true secret key $\mathsf{sk}^*$ the combiner is not correct on at least one evaluation point $x_i$. □

Finally we want to note that the distinguisher constructed in the proof of Theorem 3.3 can predict the image of any arbitrary pre-image $x \in \{0,1\}^\lambda$.

This means that the adversary is strong enough win the weak pseudorandomness and unpredictability security experiment with significant advantage. As a corollary of Theorem 3.3 we get the impossibility of robust $(n; \lfloor n/2 \rfloor)$ black-box combiners for the VUF and WVRF primitive.

**Corollary 3.18 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VUF-combiner)** *There exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the VUF primitive as defined in Definition 2.11.*

**Corollary 3.19 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box WVRF-combiner)** *There exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the WVRF primitive as defined in Definition 2.10.*

## 3.3 Strong Robust Black-Box Combiners

Having ruled out the possibility of combining VRFs, VUFs, wVRFs even for statistical notions of correctness and unique provability and all oracles pseudorandomness we need to make an even stronger assumption to allow for combiners to exist. We will assume that all candidates satisfy unique provability. This is a strong assumption since unique provability is one of the main properties of VRFs which distinguishes VRFs from PRFs which admit simple robust $(n;1)$ black-box combiner constructions. We note that unique provability is generally required to be perfect for good constructions, meaning that most existing constructions come with rigorous mathematical proofs that assert their unique provability. Examples of this are ECVRF and Lattice VRFs, which are the most famous constructions of VRFs.

change

As previously mentioned, PRFs admit a simple robust $(n;1)$ black-box combiner which simply xors the outputs of all PRF candidates. We show that this construction can be extended to VRFs by simply computing the trace of the evaluation algorithms trace and providing this as a proof. The existence of such combiners is formalized in the following lemma.

**Lemma 3.20 (Strong Robust $(n;1)$ Black-Box VRF Combiners)** *There exist robust $(n;1)$ black-box combiners in the sense of Definition 2.12 for the VRF primitive Definition 2.9 if we assume that all candidates are uniquely provable and perfectly correct.*

**Proof of Lemma 3.20.** We show the existence of a robust $(2;1)$ black-box VRF-combiner under the assumption that all candidates are perfectly correct and uniquely provable. A robust $(n;n-1)$ black-box VRF-combiner can be constructed by iteratively applying the construction on the previous combiner and a new candidate instance until all candidates are combined. We provide a construction which is very similar to PRF combiners shown in Section 1.2, the key generation function samples a key pair for both candidates and returns a secret key $\mathsf{sk} = (\mathsf{sk}_1, \mathsf{sk}_2)$ and public key $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$. The evaluation algorithm computes the image by computing the XOR of the individual outputs $y = y_1 \oplus y_2$ and the proof by collecting all intermediate results $\pi = (y_1, \pi_1, y_2, \pi_2)$. The verification then works by first checking whether both the values $y_1, y_2$ have been correctly computed with $\mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x_1, y_1, \pi_1)$ and $\mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x_2, y_2, \pi_2)$ and then subsequently checks whether $y = y_1 \oplus y_2$. The combiner construction is shown in Figure 3.4.

**Claim 3.21** *The combiner in* **??** *is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5 if all candidates are perfectly correct Definition 2.4 and uniquely provable Definition 2.5.*

The proof of this claim follows essentially by definition of the combiner construction and the assumed unique provability and pseudorandomness of

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$

1 : $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow_\$ \mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
2 : $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow_\$ \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : **return** $((\mathsf{sk}_1, \mathsf{sk}_2), (\mathsf{pk}_1, \mathsf{pk}_2))$

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** $(y, \pi)$

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : **req** $y = y_1 \oplus y_2$
4 : **req** $\mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1) = 1$
5 : **req** $\mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2) = 1$

**Figure 3.4:** Combiner for VRFs if all candidate schemes are uniquely provable.

all candidates. We lead the detailed argument in the appendix but believe that its easy to see why the claim holds. We only need to show that the combiner is pseudorandom, we employ a reductionist approach to show the following claim.

**Claim 3.22** *The combiner in* **??** *is* $(t, \varepsilon)$*-pseudorandom in the sense of Definition 2.6 if at least one candidates are* $(t, \varepsilon)$*-pseudorandom. In particular if at least one candidate is pseudorandom then the combiner is pseudorandom.*

Assume, towards a contradiction, that there exists an adversary $\mathcal{A}$ running in time $t$ that breaks pseudorandomness of the combiner $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ with advantage $\varepsilon$, we show that we can construct an adversary $\mathcal{R}_i$ running in time $t_i$ that wins the prf security experiment of every candidate with advantage $\varepsilon_i$. The reduction $\mathcal{R}_i$ simulates all the remaining candidates queries and uses adversary $\mathcal{A}$ to break that one candidate. We show the reduction $\mathcal{R}_i$ in Figure 3.5.

finish the reduction

Having shown that general notion of combiners on VRFs even with relaxed requirements on the security properties of the combiner are impossibile and the combiners on VRFs with guaranteed unique provability are easy we want to strengthen the result by considering combiners for VRFs (assuming all candidates are uniquely provable) with advanced additional properties, namely constrainability as in **??**, key-homomorphism as in **??**, aggregability as in **??** and finally unbiasability as in **??**.

Reduction $\mathcal{R}_i^{\mathcal{O}^{prf}_{eval,i}}(\mathsf{pk}_i)$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}_{1-i}, \mathsf{pk}_{1-i}) \leftarrow_{\$} \mathcal{O}^{prf}_{eval,1-i}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : $\mathsf{sk} \leftarrow (\mathsf{sk}_0, \mathsf{sk}_1)$
4 : $\mathsf{pk} \leftarrow (\mathsf{pk}_0, \mathsf{pk}_1)$
5 :

**Figure 3.5:** Reduction.

Finally we want to mention that more advanced notions of VRF such as constrainable VRFs [**?**] which additionally allow the constraining of secret key to a specific subset of evaluation points. Key-homomorphic VRFs [**?**] satisfy that the secret key space $\mathcal{SK}$ and image-proof space $\mathcal{Y} \times \mathcal{P}$ admit algebraic group structures $(\mathcal{SK}, \circledast_{\mathcal{SK}})$ and $(\mathcal{Y} \times \mathcal{P}, \circledast_{\mathcal{Y}\times\mathcal{P}})$ and $\mathsf{Eval}_{\mathsf{VRF}}(\cdot, x)$ admits a group homomorphism for any $x \in \{0,1\}^\lambda$. Aggregable VRFs [**?**] additionally implement algorithms to aggregate many public keys and corresponding image-proof pairs into a single image-proof pair, requiring that in addition to the existing properties of the VRF the aggregated input is pseudorandom and statistically binding. Finally unbiasable VRFs [**?**] are VRFs which additionally protect from adversaries skewing the output distribution of the VRF by maliciously crafting public keys. We show that under the assumption of uniquely provable and perfectly correct candidates all of those advanced notions admit robust $(2;1)$ black-box combiners.

## 3.4 Discussion

There are different ways to combine VRFs, for both unique provability and key-homomorphism they cannot be combined in a black-box fashion if we want to preserve properties such as unpredictability and therefore also randomness. On the other hand we manage to construct a robust black-box combiner that combines unbiasabile VRFs under guaranteed unique provability for all candidates. We find that the black-box combination of information theoretic properties generally seems hard if other properties such as unpredictability need to be preserved. While the class of all robust black-box combiners initially seems to be very broad we show that we can capture all polynomially bounded combiners using the notion of a robust black-box DAG combiner, which allows a more systematic analysis of possible constructions by abstracting a graph structure in the computation executed by the implemented algorithms. We have not considered possible combiner constructions for stronger relaxations, such as computational unique provability or correctness with error, this is mainly because the resulting construction only offer weak guarantees that are insufficient for practical applications.

**Extensions.** We show that there exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner for the VRF primitive, naturally we wonder whether an impossibility can be shown for robust $(n; n-1)$ black-box VRF-combiners which eliminates the possibility of combining VRFs if even one candidate is not guaranteed to be secure. We believe this statement to be true. Additionally it would be interesting to study how to strongly combine the notion of unbiasable VRFs, another advanced notion of VRFs introduced in [**?**].

Chapter 4

# Conclusion

# Bibliography

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986.

[MRV99] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. Verifiable random functions. pages 120–130, 1999.

# Appendix

## The Difference Lemma

We present the proof of the difference lemma, formally:

**Lemma .1 (Difference Lemma)** *Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events with*

$$\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$$

*Then we can bound the difference in probability of $\mathcal{W}_0$ and $\mathcal{W}_1$ with*

$$|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| \leq \Pr[\mathcal{E}]$$

**Proof of Lemma .1.** Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events and assume $\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$. Then we can deduce the following:

$$\begin{aligned}
&|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| && (.1)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] + \Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]| && (.2)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}]| && (.3)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}]\Pr[\mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]\Pr[\mathcal{E}]| && (.4)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]|\Pr[\mathcal{E}] && (.5)\\
\leq&\Pr[\mathcal{E}] && (.6)
\end{aligned}$$

## Conjecture

We quickly want to make a few remarks towards the idea of extending the proof of **??** to the more general case of **??**. The direct proof fails because of the subtle reason that the hidden oracle queries need to be candidate-wide irrelevant but not jointly irrelevant over queries to many distinct candidate schemes. In the proof of **??** we argue that for oracle $\mathcal{O}_2$ all hidden oracle queries, i.e. all oracle queries not contained in the transcript of the combiners verification algorithm are exactly the queries that would influence the combiner output if changes. On the other hand, for any arbitrary assignment of

the hidden queries, the output of the combiner stays constant as long as the non-hidden queries and the queries to $\mathcal{O}_1$ are honestly answered.

This idea gives rise to the attack of iteratively filtering the secret key space on a polynomial number of points by keeping in every round $(1.)$ at least the true secret key, $(2.)$ only secret keys for which the output of the combiner is the same as for the true secret key. We argue that after a sufficient number of filtering steps, any remaining candidate has sufficiently high chance of evaluating to the same output as the combiner does with the true secret key on a uniformly random evaluation point.

However, since the hidden queries must not necessarily be jointly-irrelevant this argument cannot be applied if we have more candidates, of which still $n - 1$ need to be secure for the combiner to be required to implement unique provability and pseudorandomness.

TODO example of axis-constant function

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

**Title of paper or thesis**:

**Authored by**:
*If the work was compiled in a group, the names of all authors are required.*

| **Last name(s):** | **First name(s):** |
|---|---|
| | |
| | |
| | |
| | |

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

| **Place, date** | **Signature(s)** |
|---|---|
| | |
| | |
| | |
| | |

*If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.*

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).